

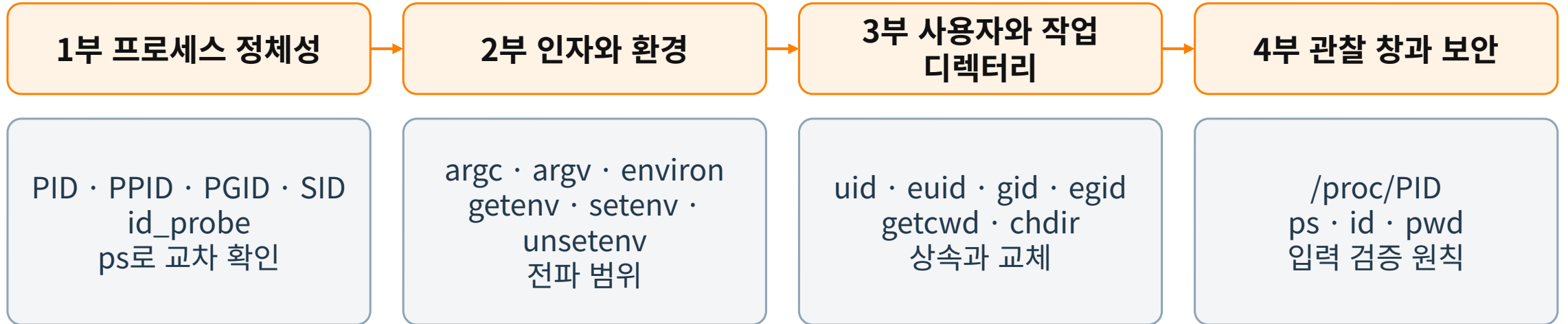
CSE232 시스템프로그래밍(캡스톤디자인)

6주차 수업

프로세스 정체성 · 인자 · 환경

경민기

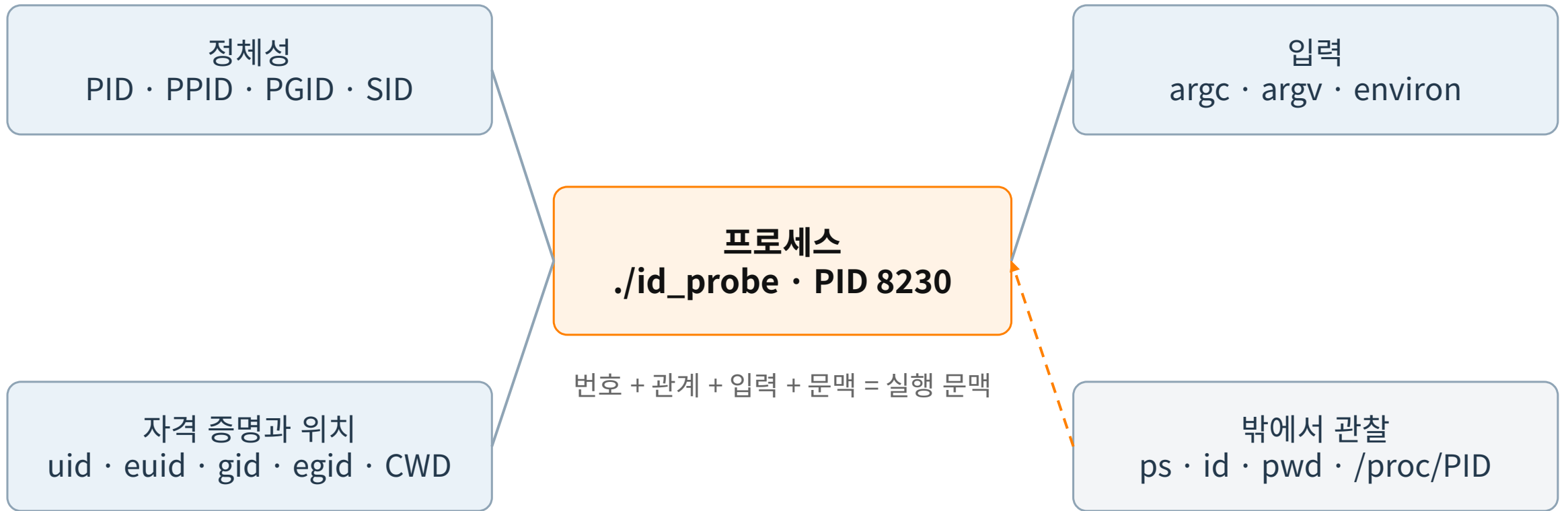
오늘의 지도



반복 동작: 프로그램이 스스로 보고한 값 ↔ ps · id · pwd · /proc가 밖에서 관찰한 값

모든 조회는 즉시 답을 주는 호출입니다. 반환값 검사가 전부입니다.

프로세스의 실행 문맥



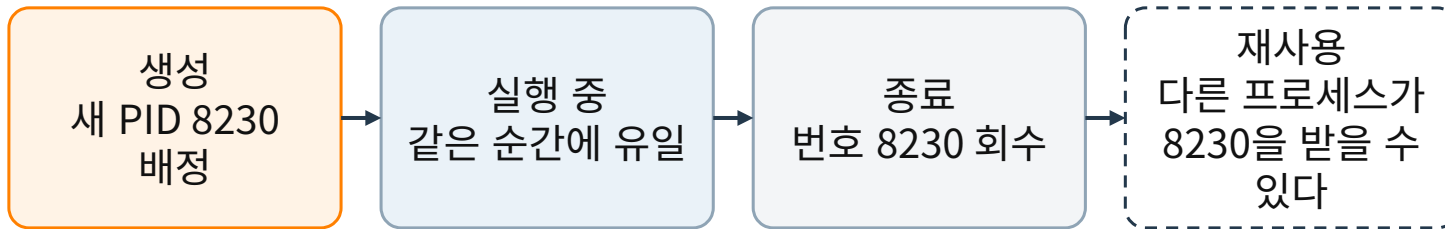
5주차는 바깥(uname·sysconf·getrlimit)을 물었고, 오늘은 처음으로 프로세스 자신을 묻습니다.

1부

프로세스 정체성

PID · PPID · PGID · SID 관계를 확인합니다.

PID: 프로세스 번호



PID는 자원이 아니라 이름입니다. 오래된 PID를 저장해 두고 나중에 같은 프로세스라고 가정하면 안 됩니다.

```
pid_t pid = getpid(); /* 반환값 검사 없음 */  
printf("pid=%ld\n", (long)pid);
```

헤더 <unistd.h> · 반환형 pid_t · getpid는 실패 경로가 정의되지 않은 드문 호출입니다.

커널의 식별자

프로세스를 구분하는 양의 정수다. 같은 순간에 중복되지 않는다.

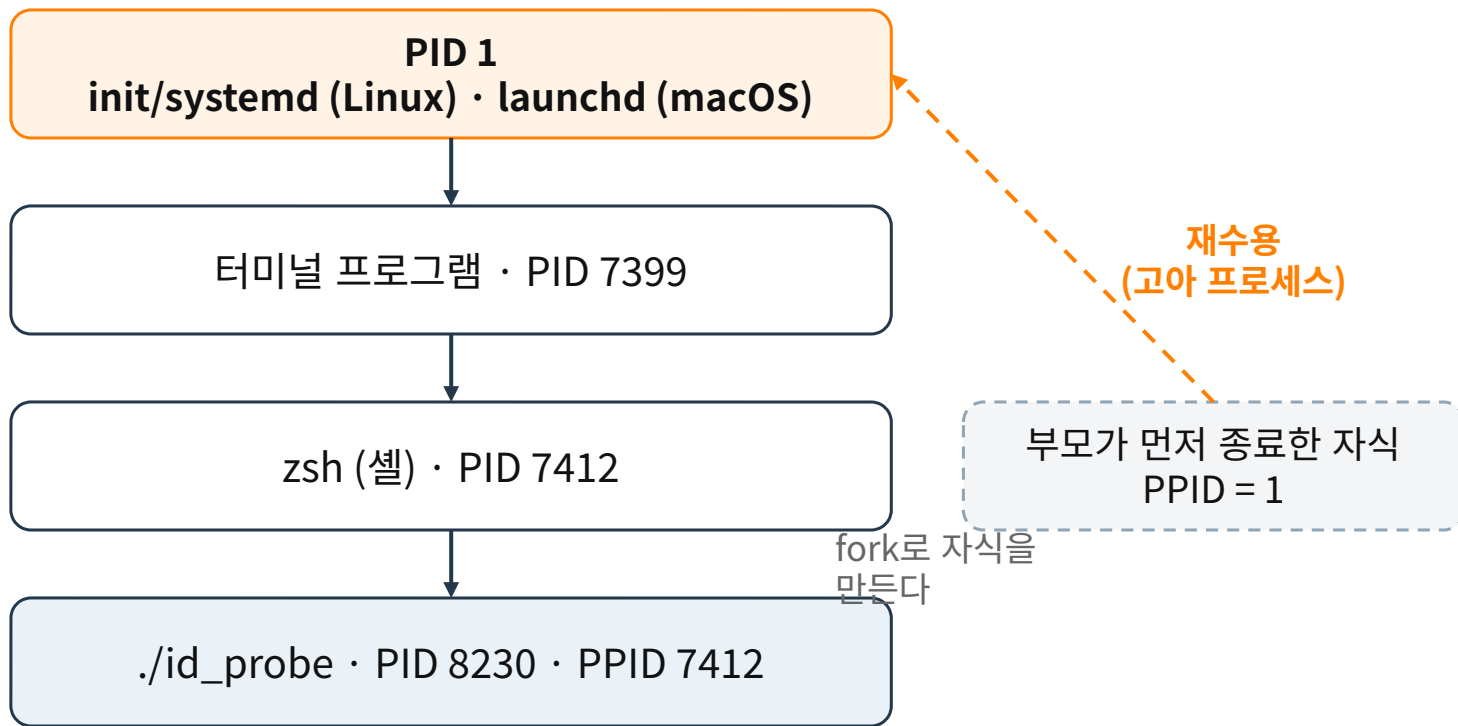
생성과 소멸

새 프로세스에 새 PID가 붙고 종료하면 번호는 회수된다.

재사용

회수된 번호는 나중에 다른 프로세스에 다시 쓰일 수 있다.

PPID와 프로세스 트리



부모 프로세스

PPID는 나를 실행한 프로세스다. 터미널에서는 대개 셸이다.

트리 구조

모든 프로세스는 부모를 거슬러 올라가 PID 1에 닿는다.

고아 재수용

부모가 먼저 종료하면 자식은 PID 1 아래로 재수용된다.

부모-자식 관계는 만들어진 시점의 사실입니다. `ps tree -p(Linux)` 또는 `ps -ef`로 확인합니다.

id_probe: 정체성 조회

```
printf("pid=%ld\n", (long)getpid());
printf("ppid=%ld\n", (long)getppid());
printf("pgid=%ld\n", (long)getpgrp());
printf("sid=%ld\n", (long)getsid(0));

const char *tty = ttyname(STDIN_FILENO);
if (tty == NULL) {
    printf("tty=(none)\n");
} else {
    printf("tty=%s\n", tty);
}
```

네 가지 조회

모두 인자 없이 pid_t를 돌려준다. getsid(0)의 0은 자기 자신이다.

실패 경로

네 조회는 실패를 정의하지 않는다. getsid도 0을 넘기는 오늘날의 용법은 안전하다.

tty 조건

ttyname은 표준 입력이 터미널이 아니면 NULL이다. 오류가 아니라 관찰 결과다.

id_probe.c · 1-17행 · 헤더와 정체성 조회

```
01 #include <stdio.h>
02 #include <sys/types.h>
03 #include <unistd.h>
04
05 /* 오늘의 정체성 조회 네 개는 모두 인자 없이 pid_t를 돌려주고
06    실패 경로를 정의하지 않는다. pid_t의 크기는 시스템마다 다를
07    수 있으므로 출력 전에 (long)으로 변환한다. */
08 int main(int argc, char **argv)
09 {
10     if (argc != 1) {
11         fprintf(stderr, "usage: %s\n", argv[0]);
12         return 2;
13     }
14     printf("pid=%ld\n", (long)getpid());
15     printf("ppid=%ld\n", (long)getppid());
16     printf("pgid=%ld\n", (long)getpgrp());
17     printf("sid=%ld\n", (long)getsid(0));
```

1-3행 · 헤더

unistd.h에 오늘의 조회가 모두 선언되어 있다.

5-7행 · 파일 주석

네 조회의 공통 계약과 (long) 변환 이유를 적었다.

10-17행 · 진입과 조회

인자 검사 뒤 네 정체성을 같은 서식으로 출력한다.

id_probe.c · 18-33행 · tty 관찰과 종료

```
18  /* 표준 입력이 터미널이 아니면(입력 리다이렉션·크론 등)
19     ttyname이 NULL을 돌려준다. 이것도 관찰 결과다. */
20  const char *tty = ttyname(STDIN_FILENO);
21  if (tty == NULL) {
22      printf("tty=(none)\n");
23  } else {
24      printf("tty=%s\n", tty);
25  }
26  printf("compare: ps -o pid,ppid,pgid,tty,stat,command -p %ld\n",
27        (long)getpid());
28  if (fflush(stdout) == EOF) {
29      perror("stdout");
30      return 1;
31  }
32  return 0;
33 }
```

18-20행 · tty 조회

표준 입력의 터미널 이름을 묻는다. 입력이 터미널이 아니면 NULL이다.

21-25행 · NULL 분기

없다는 결과도 (none)으로 출력에 남긴다.

26-27행 · 교차 확인 안내

ps 비교 명령을 프로그램이 알려 준다. Linux는 sid 열을 더해 볼 수 있다.

PGID와 세션: 작업 제어의 축

세션 SID 7412 · 제어 터미널 /dev/pts/0 (로그인 셸이 세션 대표)

그룹 7412 · 셸

zsh
PID 7412 · PGID 7412

포그라운드 그룹 8300 · `cat f | grep x | wc`

cat f
PID 8300
PGID 8300

grep x
PID 8301
PGID 8300

wc
PID 8302
PGID 8300

백그라운드 그룹 8310

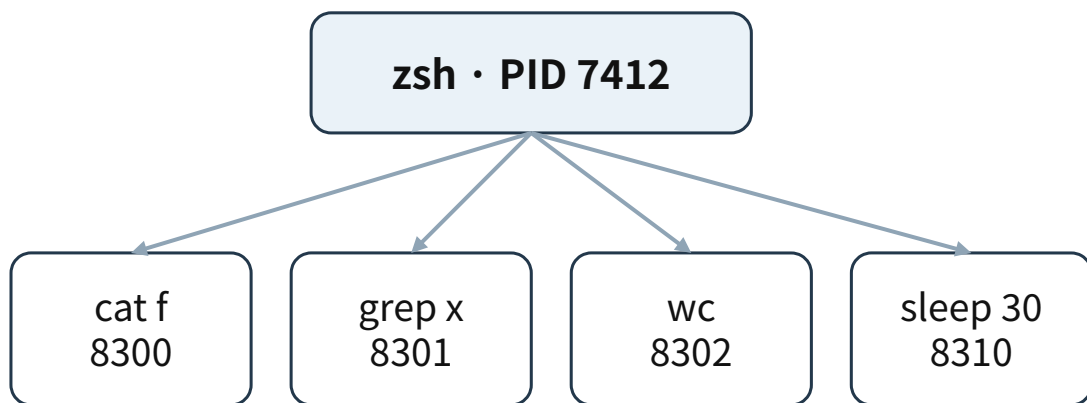
sleep 30 &
PID 8310 · PGID 8310

↑
Ctrl+C → 포그라운드 그룹 전체

대화형 셸은 작업(job)마다 새 그룹을 만듭니다. 그룹 대표(첫 프로세스)의 PID가 그 그룹의 PGID입니다.

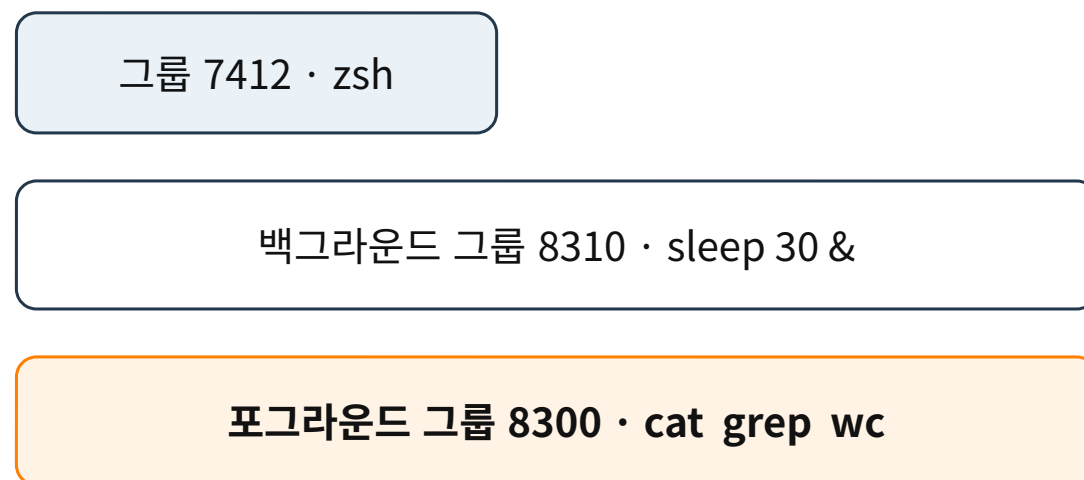
두 축: 트리와 작업 제어

부모-자식 관계 (PPID) · 태어난 관계



네 프로세스 모두 PPID = 7412.
트리만 보면 넷은 구분되지 않습니다.

그룹·세션 (PGID·SID) · 작업 제어의 관계



↑
Ctrl+C (SIGINT)는 포그라운드 그룹 전체에 갑니다
모두 같은 세션 7412 · 같은 제어 터미널

ttynum(3)은 표준 입력에 연결된 터미널 이름을 돌려줍니다. 터미널이 아니면 NULL입니다.

ps로 교차 확인

```
$ ps -o pid,ppid,pgid,sid,tt,stat,cmd -p $$
  PID   PPID   PGID   SID TT    STAT CMD
  7412   7399   7412   7412 pts/0  Ss   -bash

$ sleep 30 &
$ ps -o pid,ppid,pgid,sid,tt,stat,cmd -p $!
  PID   PPID   PGID   SID TT    STAT CMD
  8310   7412   8310   7412 pts/0  S     sleep 30
```

같은 값 확인

id_probe가 보고한 값과 ps의 같은 열을 맞춘다.
\$\$는 셸, \$!는 직전 작업이다.

새 그룹

셸이 만든 자식이라 PPID는 셸의 PID다. 작업마다 새 그룹이 생겨 PGID는 자기 PID다.

macOS 차이

macOS ps에는 sid·cmd 열이 없다. tty·command를 쓰고 SID는 id_probe끼리 비교한다.

출력 예시: Linux(procps) · bash 환경입니다.

직접 확인 1: 정체성

```
./id_probe
./id_probe < /dev/null      # 표준 입력이 터미널이 아니면 tty=(none)

ps -o pid,ppid,pgid,sid,tty,stat,cmd -p $$      # Linux
ps -o pid,ppid,pgid,tty,stat,command -p $$      # macOS

sleep 30 &
ps -o pid,ppid,pgid,command -p $!
kill %1

pstree -p | less           # Linux · macOS는 ps -ef
```

sleep 30이 살아 있는 동안 ps로 확인하고 kill %1로 정리합니다.

두 문 확인

id_probe 출력과 ps의 같은 열을 비교한다.

& 실험

백그라운드 자식의 PPID는 셸, PGID는 자기 PID다.

트리 관찰

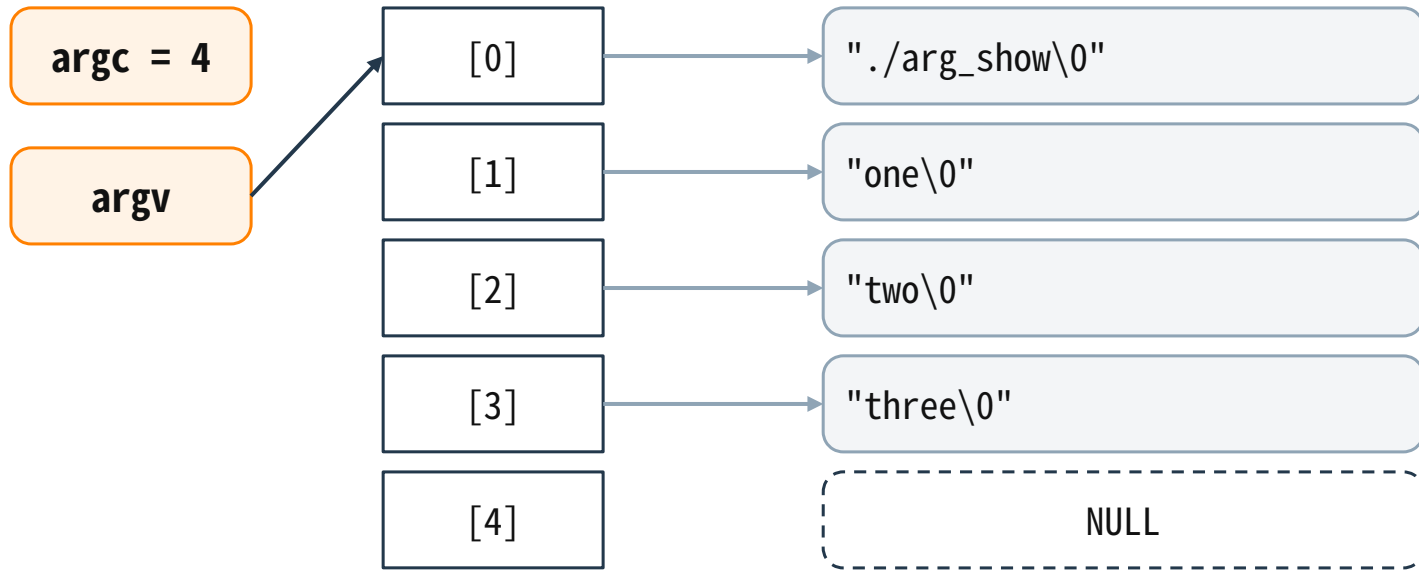
pstree 또는 ps -ef에서 자기 셸과 부모를 찾는다.

2부

인자와 환경

argc·argv와 environ의 구조와 수명을 다룹니다.

argc · argv의 구조



각 문자열은 \0으로 끝나고, 배열은 NULL로 끝납니다. 두 번째 끝 표시 덕분에 argc 없이도 순회할 수 있습니다.

포인터 배열

argv는 널 종료 문자열을 가리키는 포인터 배열이다.

끝 표시

argv[argc]는 NULL이 보장된다. 개수와 NULL, 두 가지 끝 표시가 있다.

인덱스 의미

argv[0]은 관례상 프로그램 이름이지만 보장은 아니다.

`./arg_show one two three` 실행 시의 모습 · `char *argv[]` · `argv[argc] == NULL`

arg_show: 인자 순회

```
printf("argc=%d\n", argc);
for (int i = 0; i < argc; i++) {
    printf("argv[%d]=%s\n", i, argv[i]);
}
printf("argv[%d]=%p\n", argc, (void *)argv[argc]);
```

개수 우선

argc를 먼저 확보하면
반복 범위가 명확해진다.

NULL 확인

argv[argc]가 NULL임을
직접 관찰한다.

입력 검증

받은 문자열의 길이·형식은
사용하기 전에 검증한다.

arg_show.c · 1-21행 · 인자 배열 관찰

```
01 #include <stdio.h>
02
03 /* argv는 널 종료 문자열의 포인터 배열이다.
04    argv[argc]는 NULL이 보장되므로 개수와 널 두 가지로
05    끝을 확인할 수 있다. argv[0]은 관례상 프로그램 이름이지만
06    호출 쪽이 무엇이든 넣을 수 있으므로 보증으로 쓰지 않는다. */
07 int main(int argc, char **argv)
08 {
09     printf("argc=%d\n", argc);
10     for (int i = 0; i < argc; i++) {
11         printf("argv[%d]=%s\n", i, argv[i]);
12     }
13     printf("argv[%d]=%p\n", argc, (void *)argv[argc]);
14     /* exec -a 등으로 argv[0]을 바꾼 실행과 비교할 때
15        문자열 길이도 함께 출력하면 길이 검증의 출발점이 된다. */
16     if (fflush(stdout) == EOF) {
17         perror("stdout");
18         return 1;
19     }
20     return 0;
21 }
```

3-6행 · 파일 주석

배열 구조와 argv[0]의
비보증을 먼저 적었다.

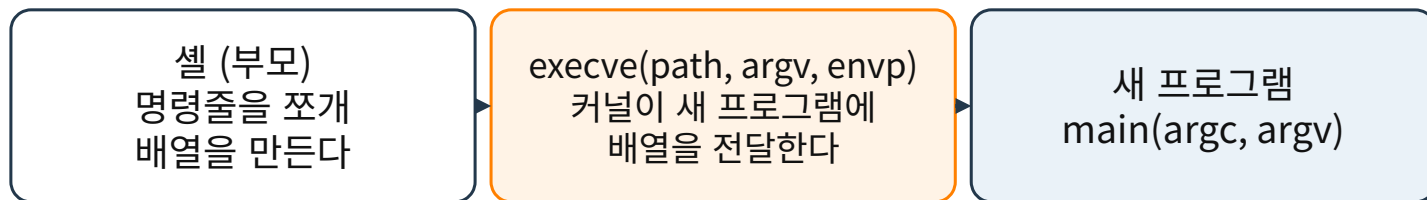
9-13행 · 관찰

argc, 각 인자, 그리고
argv[argc]의 NULL을 출력한다.

14-15행 · 주석

argv[0] 교체 실험과
길이 검증으로 이어지는 힌트다.

argv는 어디서 오는가



```
$ bash -c 'exec -a fake-name ./arg_show x'
```

```
argc=2
argv[0]=fake-name
argv[1]=x
argv[2]=0x0      (Linux: argv[2]=(nil))
```

실행하는 쪽이 어떤 문자열이든 넣을 수 있습니다. argv[0]은 관례일 뿐 보증이 아닙니다.

argv[0]은 로그·오류 메시지 표기로만 쓰고 프로그램 판별의 근거로는 쓰지 않습니다.

보증이 아니다

셸은 보통 실행한 경로를 argv[0]에 넣지만 `exec -a`로 얼마든지 바꿀 수 있다.

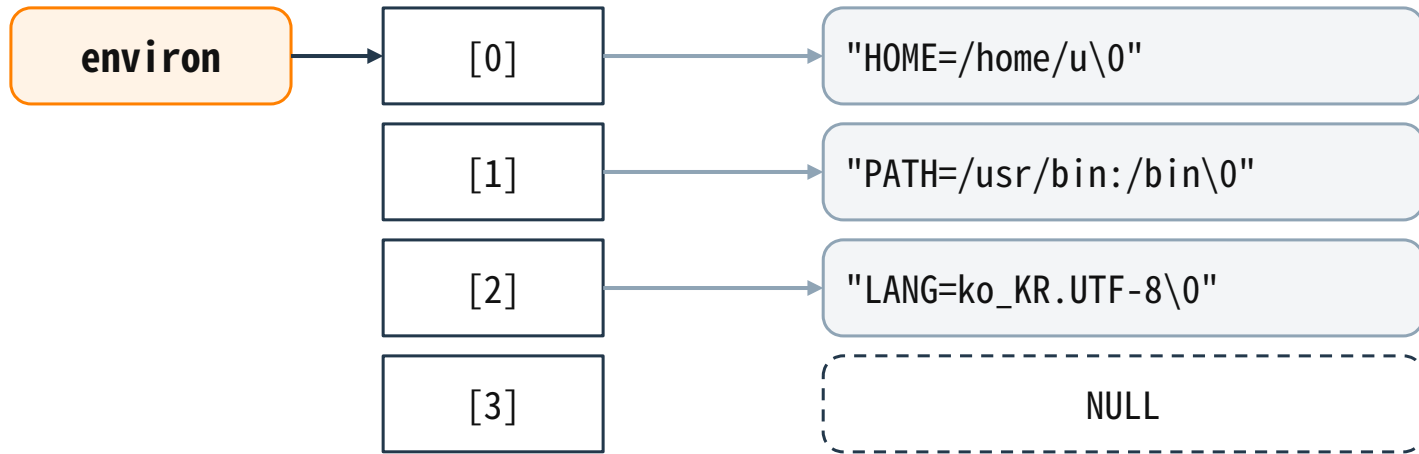
ps도 argv를 본다

관리자가 ps로 보는 명령줄이 argv다. 인자로 넘긴 비밀은 그대로 드러난다.

ARG_MAX

인자와 환경의 총 크기 상한.
5주차 sysconf로 조회했다.

environ: 같은 모양의 배열



```
extern char **environ;  
for (char **e = environ; *e != NULL; e++)  
    printf("%s\n", *e);
```

구조

"NAME=value" 문자열의 널 종료 포인터 배열이다. 순회 코드가 argv와 같다.

선언

`extern char **environ;`을 프로그램이 직접 선언한다. 헤더 선언은 조건부다.

시작 값

부모 프로세스 환경의 복사본으로 시작한다.

glibc는 `_GNU_SOURCE`를 정의할 때만 `unistd.h`가 `environ`을 선언합니다. `main`의 `envp`는 POSIX 표준이 아닙니다.

env_dump: 환경 순회

```
extern char **environ;

static int count_env(void)
{
    int n = 0;
    for (char **e = environ; *e != NULL; e++) {
        n++;
    }
    return n;
}
```

같은 패턴

argv 순회와 같은 널 종료 배열 순회다.

요소 형태

각 요소는 "이름=값" 문자열 하나다. dump_env가 같은 순회로 *e를 출력한다.

개수 관찰

setenv·unsetenv 뒤 개수 변화를 비교한다.

env_dump.c · 1-17행 · 헤더와 count_env

```
01 #include <stdio.h>
02 #include <stdlib.h>
03 #include <string.h>
04
05 /* 환경은 "NAME=value" 문자열의 널 종료 포인터 배열이다.
06     environ 전역 변수로 순회하고 getenv·setenv·unsetenv로
07     조회·변경하는 계약을 관찰한다. */
08 extern char **environ;
09
10 static int count_env(void)
11 {
12     int n = 0;
13     for (char **e = environ; *e != NULL; e++) {
14         n++;
15     }
16     return n;
17 }
```

1-3행 · 헤더

stdlib.h는 getenv·setenv,
string.h는 strcmp·strchr용이다.

8행 · environ 선언

런타임이 정의한 전역 배열을
extern으로 가져다 쓴다.

10-17행 · count_env

argv와 같은 널 종료 순회로
개수를 센다.

env_dump.c · 18–35행 · dump_env와 find_env

```
18
19 static void dump_env(void)
20 {
21     for (char **e = environ; *e != NULL; e++) {
22         printf("%s\n", *e);
23     }
24 }
25
26 static int find_env(const char *name)
27 {
28     const char *value = getenv(name);
29     if (value == NULL) {
30         printf("%s=(unset)\n", name);
31     } else {
32         printf("%s=%s\n", name, value);
33     }
34     return 0;
35 }
```

19–24행 · dump_env

전체 항목을 한 줄씩
출력하기만 하는 순회다.

28행 · getenv

조회 결과는 내부 저장소를
가리키는 포인터다.

29–33행 · NULL 분기

unset을 오류와 구분해
기본 표기로 처리한다.

env_dump.c · 36–51행 · main 진입과 --find

```
36
37 int main(int argc, char **argv)
38 {
39     if (argc > 3) {
40         fprintf(stderr, "usage: %s [--find NAME | --set NAME=VALUE"
41             " | --unset NAME]\n", argv[0]);
42         return 2;
43     }
44     if (argc == 1) {
45         printf("env_count=%d\n", count_env());
46         dump_env();
47         return 0;
48     }
49     if (strcmp(argv[1], "--find") == 0 && argc == 3) {
50         return find_env(argv[2]);
51     }
```

39–43행 · 사용법 검사

인자가 많으면 안내 후
종료 상태 2다.

44–48행 · 기본 동작

인자 없이 실행하면 개수와
전체 환경을 출력한다.

49–51행 · --find

이름 하나를 getenv로
조회하는 하위 명령이다.

env_dump.c · 52-71행 · --set 블록

```
52  /* setenv의 세 번째 인자 1은 같은 이름이 있으면 덮어쓴다는 뜻이다.  
53     바꾼 뒤 개수와 값을 다시 조회해 변경이 이 프로세스 안에서만  
54     일어났음을 관찰한다. 부모 셸에는 영향을 주지 않는다. */  
55  if (strcmp(argv[1], "--set") == 0 && argc == 3) {  
56      char *eq = strchr(argv[2], '=');  
57      if (eq == NULL || eq == argv[2]) {  
58          fprintf(stderr, "--set expects NAME=VALUE\n");  
59          return 2;  
60      }  
61      size_t name_len = (size_t)(eq - argv[2]);  
62      char name[name_len + 1];  
63      memcpy(name, argv[2], name_len);  
64      name[name_len] = '\0';  
65      if (setenv(name, eq + 1, 1) == -1) {  
66          perror("setenv");  
67          return 1;  
68      }  
69      printf("env_count=%d\n", count_env());  
70      return find_env(name);  
71  }
```

56-60행 · 입력 검증

등호가 없거나 맨 앞이면
형식 오류로 종료 2다.

61-64행 · 이름 분리

NAME=VALUE에서 이름만
널 종료 문자열로 복사한다.

65-70행 · 변경·재조회

setenv 뒤 개수와 값을 다시
읽어 결과를 확인한다.

env_dump.c · 72-83행 · --unset과 마무리

```
72     if (strcmp(argv[1], "--unset") == 0 && argc == 3) {
73         if (unsetenv(argv[2]) == -1) {
74             perror("unsetenv");
75             return 1;
76         }
77         printf("env_count=%d\n", count_env());
78         return find_env(argv[2]);
79     }
80     fprintf(stderr, "usage: %s [--find NAME | --set NAME=VALUE"
81                " | --unset NAME]\n", argv[0]);
82     return 2;
83 }
```

72-76행 · --unset

삭제도 실패 경로를 갖는다:
반환값을 검사한다.

77-78행 · 재조회

지운 뒤 (unset) 표기와
개수 변화로 확인한다.

80-82행 · 마지막 usage

모든 하위 명령에 안 맞으면
사용법 오류로 끝낸다.

getenv · setenv · unsetenv 계약

호출	성공	실패 또는 없음
<code>getenv("HOME")</code>	값 포인터 내부 저장소 · 수정 · free 금지	NULL = unset 오류가 아니라 정상 결과
<code>setenv("PROBE", "1", 1)</code>	0 · 성공 overwrite=1이면 덮어쓴다	-1 · errno = EINVAL 이름이 비었거나 =를 포함
<code>unsetenv("PROBE")</code>	0 · 성공 없는 이름이어도 성공	-1 · errno = EINVAL 이름이 비었거나 =를 포함

헤더 <stdlib.h> · 변경은 이 프로세스와 이후 자식에만 적용됩니다.

안전한 getenv 패턴

```
const char *value = getenv(name);
if (value == NULL) {
    printf("%s=(unset)\n", name);
} else {
    printf("%s=%s\n", name, value);
}

if (setenv(name, eq + 1, 1) == -1) {
    perror("setenv");
    return 1;
}
```

NULL 분기

unset 상태를 오류와 구분해 기본 표기로 처리한다. (find_env, 28-33행)

실패 경로

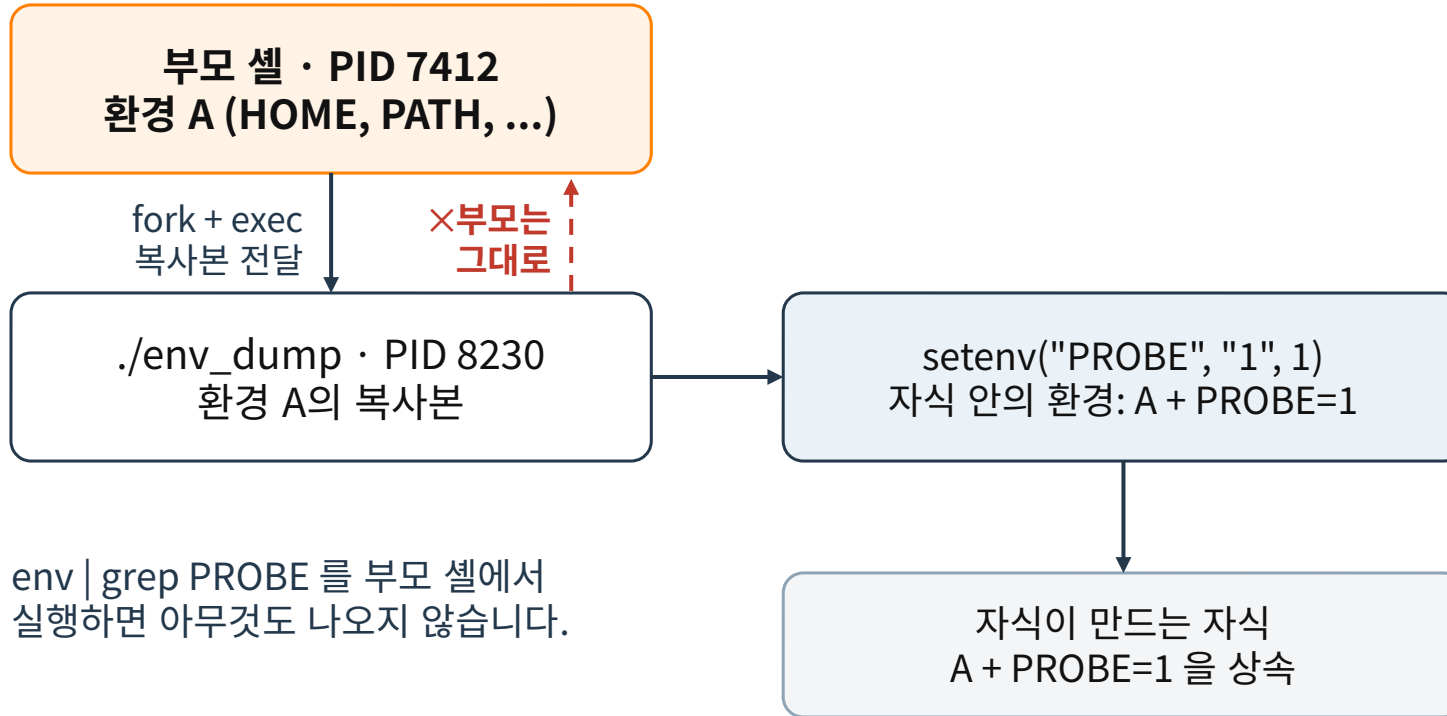
setenv는 실패해 -1을 돌려줄 수 있으므로 검사한다. (--set, 65-68행)

변경 확인

바꾼 뒤 개수와 값을 다시 조회해 확인한다. (69-70행)

완성 예제: labs/class06-01/env_dump.c · eq는 NAME=VALUE에서 = 의 위치입니다.

환경 변경의 전파 범위



한 방향

부모 → 자식으로만 흐른다.
자식은 부모를 고칠 수 없다.

export

셸의 export는 셸이 실행할
자식으로서의 상속이다.
source가 필요한 이유다.

env -i

빈 환경으로 자식을 실행하는
실험 도구다. 빈 환경도
유효한 환경이다.

MESSAGE=hi ./env_dump처럼 명령 앞 붙이기도 그 명령의 프로세스에만 적용됩니다.

직접 확인 2: 인자·환경

```
./arg_show one two three
bash -c 'exec -a fake-name ./arg_show x'

./env_dump | head -3
./env_dump --find HOME
./env_dump --find NO_SUCH_NAME
MESSAGE=hi ./env_dump --find MESSAGE
./env_dump --set PROBE=1
env | grep PROBE                # 부모 셸은 그대로
env -i PATH=/usr/bin ./env_dump
```

argv[0] 실험

exec -a로 argv[0]만 바뀐 실행을 관찰한다.

상속 관찰

명령 앞 붙이기는 그 명령에만 적용된다.

빈 환경

env -i 뒤에는 PATH 등 넘긴 항목만 남는다.

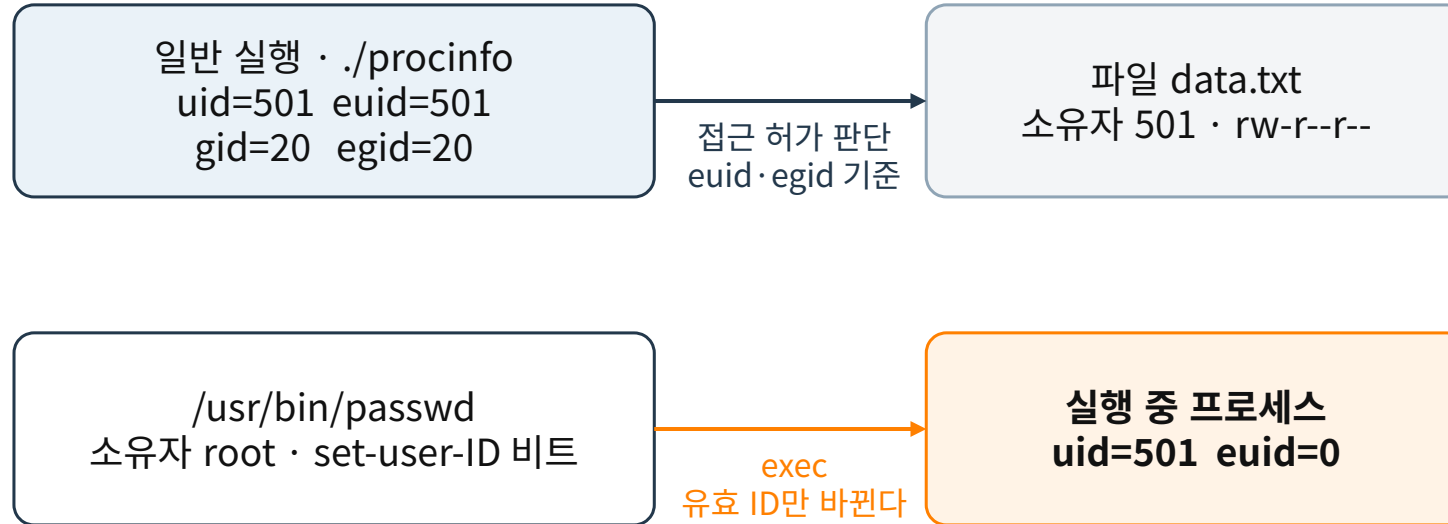
env -i ./env_dump 도 동작합니다. 빈 환경도 유효한 환경입니다.

3부

사용자와 작업 디렉터리

UID·GID와 CWD로 실행 문맥을 완성합니다.

자격 증명: uid · euid · gid · egid



일반 수업 환경에서는 uid와 euid가 같습니다. 둘을 개념적으로 나누는 이유는 set-user-ID 실행처럼 유효 ID만 바뀌는 경우가 있기 때문입니다.

실제와 유효

getuid는 실제, geteuid는 유효 사용자 ID다.

권한 판단

파일 접근 허가는 대부분 유효 ID를 기준으로 결정된다.

그룹도 같은 쌍

getgid · getegid가 같은 계약을 그룹에 대해 제공한다.

헤더 <unistd.h> · <sys/types.h> · 상세는 credentials(7)와 14주차 IPC 권한에서 이어집니다.

procinfo: 자격 증명과 문맥

```
printf("uid=%ld euid=%ld\n",
      (long)getuid(), (long)geteuid());

char cwd[4096];
if (getcwd(cwd, sizeof cwd) == NULL) {
    perror("getcwd");
    return 1;
}
printf("cwd=%s\n", cwd);
```

같은 규칙

정체성·자격 증명·위치를
같은 조회-출력 순서로 얻는다.

버퍼 조회

getcwd는 결과를 담을
버퍼를 인자로 받는다.

실패 처리

버퍼가 짧으면 ERANGE,
경로가 사라지면 ENOENT다.

procinfo.c · 1-20행 · 헤더와 count_env

```
01 #include <stdio.h>
02 #include <stdlib.h>
03 #include <string.h>
04 #include <sys/types.h>
05 #include <unistd.h>
06
07 /* 오늘 배운 실행 문맥을 한 프로그램으로 종합한다.
08    정체성(pid·ppid·pgid·sid), 자격 증명(uid·euid·gid·egid),
09    작업 디렉터리, 인자, 환경 요약을 같은 규칙으로 출력한다.
10    --env를 주면 환경 전체를, --argv를 주면 인자 전체를 나열한다. */
11 extern char **environ;
12
13 static int count_env(void)
14 {
15     int n = 0;
16     for (char **e = environ; *e != NULL; e++) {
17         n++;
18     }
19     return n;
20 }
```

1-5행 · 헤더

오늘 다룬 조희가 모두 들어 있는 예제다.

7-10행 · 파일 주석

출력 항목과 두 옵션의 역할을 요약했다.

13-20행 · count_env

env_dump와 같은 함수를 그대로 가져왔다.

procinfo.c · 21-30행 · show_env_var

```
21
22 static void show_env_var(const char *name)
23 {
24     const char *value = getenv(name);
25     if (value == NULL) {
26         printf("%s=(unset)\n", name);
27     } else {
28         printf("%s=%s\n", name, value);
29     }
30 }
```

24행 · 조회

이름을 받아 값을 얻는다:
결과는 두 갈래다.

25-29행 · NULL 분기

unset을 기본 표기로
출력하는 패턴 그대로다.

22행 · 재사용

HOME과 PATH 두 조회를
한 함수로 처리한다.

procinfo.c · 31-48행 · main과 옵션 파싱

```
31
32 int main(int argc, char **argv)
33 {
34     int show_env = 0;
35     int show_argv = 0;
36     for (int i = 1; i < argc; i++) {
37         if (strcmp(argv[i], "--env") == 0) {
38             show_env = 1;
39         } else if (strcmp(argv[i], "--argv") == 0) {
40             show_argv = 1;
41         } else if (strncmp(argv[i], "--", 2) == 0) {
42             fprintf(stderr, "usage: %s [--env] [--argv]"
43                     " [args...]\n", argv[0]);
44             return 2;
45         }
46         /* --로 시작하지 않는 인자는 명령행 입력으로 그대로 두고
47            --argv로 관찰한다. argv는 검증 대상이지 거부 대상이 아니다. */
48     }
```

34-35행 · 옵션 상태

두 플래그를 0으로 초기화하고
옵션을 만나면 켜다.

41-45행 · 알 수 없는 옵션

모르는 --옵션은 안내 후
종료 2: 인자 검증의 예다.

46-48행 · 입력 철학

일반 인자는 거부하지 않고
관찰 대상으로 남긴다.

procinfo.c · 49–58행 · 정체성과 tty

```
49     printf("pid=%ld ppid=%ld\n",
50           (long)getpid(), (long)getppid());
51     printf("pgid=%ld sid=%ld\n",
52           (long)getpgrp(), (long)getsid(0));
53     const char *tty = ttyname(STDIN_FILENO);
54     if (tty != NULL) {
55         printf("tty=%s\n", tty);
56     } else {
57         printf("tty=(none)\n");
58     }
```

49–52행 · 정체성

id_probe와 같은 조회를
짜지어 출력한다.

53–58행 · tty

NULL도 (none)으로 관찰
결과에 남긴다.

서식 통일

두 값을 한 줄에 묶어 보고서의
행 수를 줄인다.

procinfo.c · 59–68행 · 자격 증명과 작업 디렉터리

```
59     printf("uid=%ld euid=%ld\n",
60           (long)getuid(), (long)geteuid());
61     printf("gid=%ld egid=%ld\n",
62           (long)getgid(), (long)getegid());
63     char cwd[4096];
64     if (getcwd(cwd, sizeof cwd) == NULL) {
65         perror("getcwd");
66         return 1;
67     }
68     printf("cwd=%s\n", cwd);
```

59–62행 · 두 쌍

uid·euid와 gid·egid를
같은 서식으로 출력한다.

63–67행 · getcwd

버퍼 형태 호출과 실패 경로:
cwd_probe와 같은 모양이다.

68행 · 문맥 완성

CWD까지 출력되면 실행
문맥 보고서가 완성된다.

procinfo.c · 69–88행 · 인자·환경 요약과 종료

```
69     printf("argc=%d\n", argc);
70     if (show_argv) {
71         for (int i = 0; i < argc; i++) {
72             printf("argv[%d]=%s\n", i, argv[i]);
73         }
74     }
75     printf("env_count=%d\n", count_env());
76     if (show_env) {
77         for (char **e = environ; *e != NULL; e++) {
78             printf("%s\n", *e);
79         }
80     }
81     show_env_var("HOME");
82     show_env_var("PATH");
83     if (fflush(stdout) == EOF) {
84         perror("stdout");
85         return 1;
86     }
87     return 0;
88 }
```

70–74행 · --argv

플래그가 켜졌을 때만
인자 전체를 나열한다.

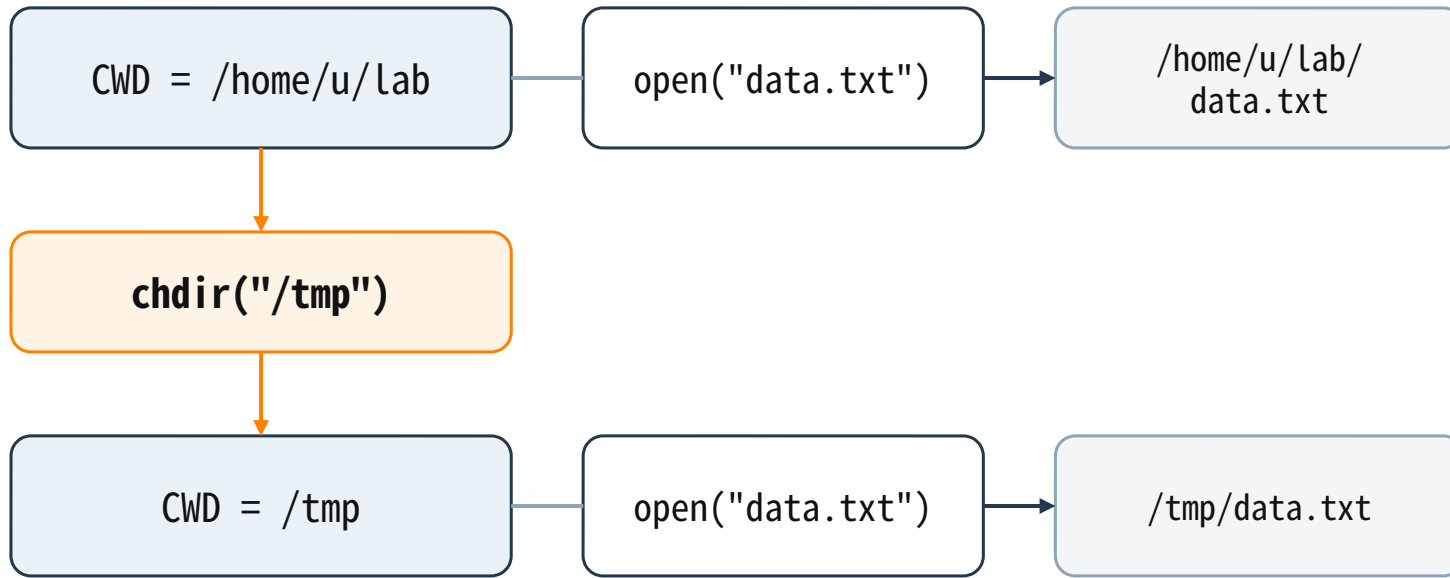
75–80행 · 환경 요약

개수는 항상, 전체 나열은
--env를 줄 때만 출력한다.

81–87행 · 마무리

HOME·PATH 조회와 버퍼 정리,
종료 상태 확정으로 끝낸다.

CWD와 상대 경로



같은 문자열 "data.txt"가 다른 파일을 가리킵니다. 로그에 `getcwd` 값을 남기면 나중에 어디서 실행했는지 재현할 수 있습니다.

질문의 의미

CWD는 상대 경로를 절대 경로로 푸는 기준이다.

두 형태

호출자 버퍼가 POSIX 기본형.
`getcwd(NULL, 0)`은 glibc·BSD 확장이며 free가 필요하다.

실패 경로

`getcwd`: `ERANGE`·`ENOENT`
`chdir`: `ENOENT`·`ENOTDIR`

`getcwd(3)` · `chdir(2)` · CWD는 프로세스별 속성이라 `chdir`은 그 프로세스 안에서만 유효합니다.

cwd_probe: chdir 실험

```
if (argc == 2) {
    if (chdir(argv[1]) == -1) {
        perror(argv[1]);
        return 1;
    }
    if (show_cwd("after_chdir") != 0) {
        return 1;
    }
}
```

실패 경로

없는 경로는 ENOENT,
파일을 가리키면 ENOTDIR이다.
perror가 경로 이름을 찍는다.

변경 확인

show_cwd가 getcwd로 실제
값을 다시 읽어 출력한다.

범위 한정

바뀐 CWD는 이 프로세스
안에서만 유효하다.

cwd_probe.c · 1-18행 · 주석과 show_cwd

```
01 #include <stdio.h>
02 #include <stdlib.h>
03 #include <unistd.h>
04
05 /* getcwd는 상대 경로를 절대 경로로 푸는 기준인 작업 디렉터리를
06    알려 준다. 호출 형태는 두 가지다. 호출자가 버퍼를 주거나,
07    (NULL, 0)으로 크기를 정해 달라고 요청하거나. chdir은 이
08    프로세스의 작업 디렉터리만 바꾼다. 부모 셸은 그대로다. */
09 static int show_cwd(const char *label)
10 {
11     char buf[4096];
12     if (getcwd(buf, sizeof buf) == NULL) {
13         perror(label);
14         return 1;
15     }
16     printf("%s=%s\n", label, buf);
17     return 0;
18 }
```

1-3행 · 헤더

stdlib.h는 show_cwd_alloc의 free를 위해 필요하다.

5-8행 · 파일 주석

두 호출 형태와 chdir의 범위를 먼저 적었다.

11-17행 · 버퍼 형태

호출자가 준 버퍼에 결과를 담고 실패는 perror로 보고한다.

cwd_probe.c · 19-30행 · show_cwd_alloc

```
19
20 static int show_cwd_alloc(const char *label)
21 {
22     char *p = getcwd(NULL, 0);
23     if (p == NULL) {
24         perror(label);
25         return 1;
26     }
27     printf("%s=%s\n", label, p);
28     free(p);
29     return 0;
30 }
```

22행 · 할당 형태

(NULL, 0)을 주면 라이브러리가 알아서 할당한다.

23-26행 · 실패 경로

NULL 반환을 검사해
perror와 종료 1로 처리한다.

27-29행 · 출력과 해제

할당받은 메모리는 출력 뒤
free로 반환한다.

cwd_probe.c · 31-43행 · main과 두 형태 실행

```
31
32 int main(int argc, char **argv)
33 {
34     if (argc > 2) {
35         fprintf(stderr, "usage: %s [directory]\n", argv[0]);
36         return 2;
37     }
38     if (show_cwd("cwd") != 0) {
39         return 1;
40     }
41     if (show_cwd_alloc("cwd_alloc") != 0) {
42         return 1;
43     }
```

34-37행 · 사용법

선택 인자는 하나까지:
그 이상이면 종료 2다.

38-43행 · 두 형태 비교

같은 값을 서로 다른 호출
형태로 두 번 읽는다.

실험 준비

인자가 있으면 다음 화면의
chdir 실험으로 진행한다.

cwd_probe.c · 44–58행 · chdir 실험과 종료

```
44     if (argc == 2) {
45         if (chdir(argv[1]) == -1) {
46             perror(argv[1]);
47             return 1;
48         }
49         if (show_cwd("after_chdir") != 0) {
50             return 1;
51         }
52     }
53     if (fflush(stdout) == EOF) {
54         perror("stdout");
55         return 1;
56     }
57     return 0;
58 }
```

44–48행 · chdir 실패

없는 경로 ENOENT, 파일이면 ENOTDIR이 perror로 나온다.

49–51행 · 변경-재조회

chdir 뒤 getcwd로 실제 값을 다시 읽는다.

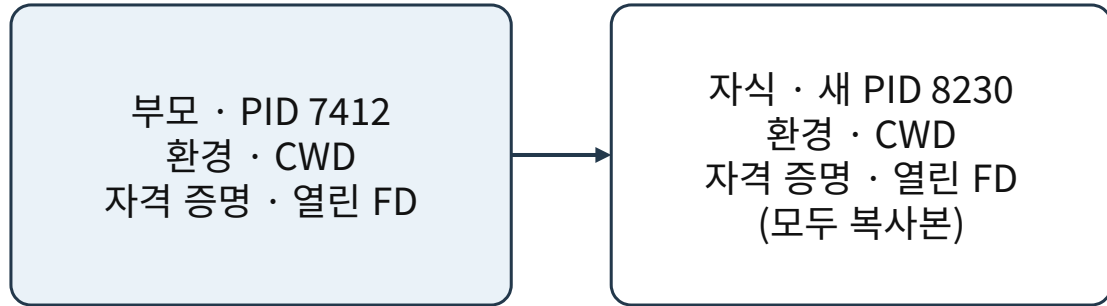
53–56행 · 종료 정리

출력 버퍼를 밀고 나서 종료 상태를 확인한다.

상속과 교체의 규칙

fork: 복제

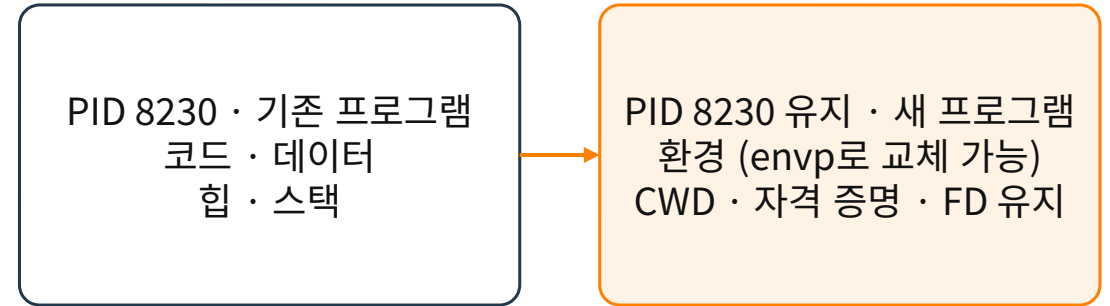
fork()



유지: 환경 · CWD · 자격 증명 · 열린 FD(기본) · PID(exec 시)

exec: 교체

execve()



바뀜: 코드 · 데이터 · 힙 · 스택 · argv·environ(새 배열을 넘기면)

셸의 cd가 서브셸 밖에 영향을 주지 않는 이유, execve에 새 환경 배열을 넘기면 환경을 통째로 바꿀 수 있는 이유가 이 그림 하나로 정리됩니다.

fork·exec 상세는 10주차에서 완성합니다.

직접 확인 3: 문맥

```
./procinfo one
./procinfo --argv one
id                                # uid·gid를 procinfo와 맞춘다
pwd

(cd /tmp && "$OLDPWD/procinfo" | grep cwd)
pwd                                # 부모 셸은 그대로

./cwd_probe /tmp
./cwd_probe /no/such/dir; echo "status=$?"
./cwd_probe /etc/hosts; echo "status=$?"
```

id 명령의 uid·gid와 procinfo 출력이 같은지 맞춥니다.

종합 보고서

procinfo 하나로 오늘의
조회 전체를 재확인한다.

CWD 실험

괄호 서브셸 안 cd는
밖의 pwd에 영향 없다.

실패 경로

없는 경로는 ENOENT,
파일 경로는 ENOTDIR이다.

4부

관찰 창과 보안

/proc와 ps로 문맥을 확인하고 신뢰 수준을 정합니다.

/proc/PID 관찰 창

/proc/8230/



Linux 전용 창입니다. /proc/self는 자기 자신입니다. 판단은 표준 API로, 진단은 /proc로 합니다.

셸 관찰 명령 (Linux)

```
grep -E '^(Name|Pid|PPid|Uid|Gid)' /proc/$$/status
tr '\0' ' ' < /proc/$$/cmdline; echo
readlink /proc/$$/cwd          # pwd·getcwd 값과 맞춘다

sleep 30 & P=$!
grep -E '^(Name|Pid|PPid)' /proc/$P/status
tr '\0' ' ' < /proc/$P/cmdline; echo
ls -l /proc/$P/fd
kill $P
```

문맥 대응

status의 Pid·PPid·Uid가
procinfo·id 출력과 같다.

인자 관찰

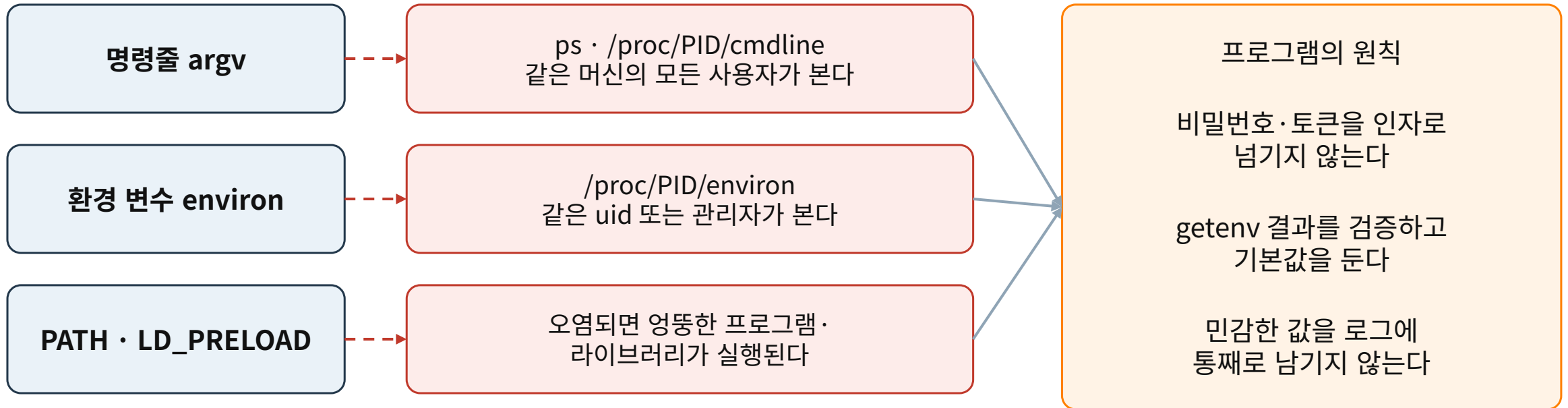
cmdline이 argv를
그대로 보여 준다.

권한 확인

다른 사용자 프로세스의
environ은 열리지 않는다.

출력 예시: Ubuntu 22.04 환경입니다.

인자·환경은 입력이다



외부에서 들어오는 값은 범위와 형식을 검증한 뒤 씁니다. 노출 경로를 알면 무엇을 인자·환경에 넣으면 안 되는지도 보입니다.

오늘의 원칙: argv와 environ은 신뢰가 아니라 입력입니다.

두 창의 교차 확인

항목	안에서 보고 · 표준 API	밖에서 관찰 · 도구
정체성	getpid · getppid · getpgrp · getsid(0)	ps -o pid,ppid,pgid,sid · /proc/PID/status
인자	argc · argv	ps -o cmd (macOS: command) · /proc/PID/cmdline
환경	environ · getenv	env · /proc/PID/environ (초기 환경)
자격 증명	getuid · geteuid · getgid · getegid	id · /proc/PID/status의 Uid·Gid
작업 디렉터리	getcwd · chdir	pwd · /proc/PID/cwd

두 창이 값이 일치하는지가 신뢰의 근거입니다. procinfo 하나로 전체를 다시 확인합니다.

정리

정체성: PID·PPID·PGID·SID로 식별한다. 트리와 작업 제어는 다른 축이다.

입력: argc·argv와 environ은 같은 모양의 배열이며 프로그램의 입력이다.

계약: getenv의 NULL은 unset이고, 변경은 이 프로세스와 자식에만 적용된다.

문맥: 자격 증명과 작업 디렉터리까지가 실행 문맥이다. chdir은 프로세스 안에서만.

확인: 프로그램의 조회를 ps·id·pwd·/proc와 교차 검증한다.

읽기: man 7 environ · man 7 credentials · 이어지는 주제: 프로세스 상태·주소 공간·자원, 9주차 시그널, 10주차 fork·exec

이것으로 할 수 있는 일

로그와 장애 추적

로그 줄마다 PID·PPID·CWD를 남기면
"누가, 어디서" 실행했는지 재현된다.
getpid · getppid · getcwd

환경 변수로 설정 받기

PORT, LOG_LEVEL 같은 설정을 getenv로
읽고, NULL이면 기본값을 쓴다.
값은 범위·형식을 검증한 뒤 사용

명령줄 도구 만들기

argv를 검증하고 usage와 종료 상태 2를
돌려주는 관례. argv[0]은 표기용.
비밀은 인자가 아니라 파일·FD로

미니 셸 (캡스톤)

작업마다 새 프로세스 그룹을 만들고
Ctrl+C를 포그라운드 그룹에 보낸다.
cd가 내장 명령이어야 하는 이유: chdir

데몬·서비스

세션을 분리하고(setsid) chdir("/")와
환경 정리로 터미널에서 독립한다.
부모가 끝나면 PID 1 아래로 재수용

프로세스 관찰·점검 도구

/proc/PID를 읽어 미니 ps·top을 만들고,
인자에 비밀을 노출하는 프로그램이나
PATH 오염을 점검한다

9주차 시그널 → 10주차 fork·exec → 캡스톤 미니 셸로 이어집니다. 오늘의 조회가 그 모든 코드의 첫 줄입니다.